

Assignment 1: Transformers from Scratch, Architectural Variants, and Byte Latent Transformers (BLT)

Course: Advanced Natural Language Processing

Deadline: September 2nd, 2026, 23:59 IST

General Instructions

1. **Implementation:** This assignment must be written in Python using PyTorch. You need to build the core transformer components from scratch. Do not use readily available PyTorch transformer modules like `nn.Transformer` or `nn.MultiheadAttention`.
 2. **AI Tools:** You can use AI tools to help debug or explain concepts, as long as your code meets the assignment requirements. Do not use AI to write your final report.
 3. **Logging:** Use Weights & Biases (WandB) to log your training runs, and host your pre-trained model checkpoints on Hugging Face. You will need to include links to both in your report.
 4. **Submission:** Submit a single `.zip` file on Moodle containing your code, outputs, and PDF report. Follow the directory structure shown in Section 5. We will not accept GitHub links or email submissions.
-

1 Overview

For this assignment, you will work with a dataset of encrypted binary sequences mapped to plaintext data. Your goal is to build a custom Sequence-to-Sequence Transformer to learn this mapping and to test how different architectural choices affect model performance.

You will focus on two main areas:

1. Building a baseline transformer from fundamental PyTorch operations.
2. Running a controlled ablation study across five configurations to evaluate the impact of positional encodings, attention mechanisms, normalization methods, and tokenization strategies.

You have to use this dataset for the assignment: [link](#).

2 Task 1: Transformer Architecture

You need to design and implement a full Encoder-Decoder Transformer using basic PyTorch operations. Write custom `nn.Module` classes for the following:

- Scaled Dot-Product Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Multi-Head Attention (MHA) and Grouped-Query Attention (GQA) blocks.
- Position-wise Feed-Forward Networks (FFN).
- Pre-Layer Normalization and RMS Normalization modules.
- Local Encoder/Decoder patch modules (specifically for the Byte Latent Transformer approach).

3 Task 2: Architectural Ablation Study

To understand how individual components impact the model, you will run a controlled experiment. You will start with a base transformer (C1) and create four variations. Each variation will change exactly one component from the base model, resulting in five total configurations.

Config	Change from Base	Positional Encoding	Attention Mechanism	Normalization	Tokenization
C1	None (Base)	Sinusoidal Absolute	Multi-Head Attention	LayerNorm	Standard Subword
C2	Positional encoding	RoPE	Multi-Head Attention	LayerNorm	Standard Subword
C3	Attention mechanism	Sinusoidal Absolute	Grouped-Query	LayerNorm	Standard Subword
C4	Normalization	Sinusoidal Absolute	Multi-Head Attention	RMSNorm	Standard Subword
C5	Tokenization	Sinusoidal Absolute	Multi-Head Attention	LayerNorm	BLT (Token-Free)

Table 1: Five architectural configurations. C2 through C5 each differ from the base model by exactly one component (highlighted in bold).

By changing only one feature at a time, you can measure the specific impact of RoPE, GQA, RMSNorm, and Token-Free processing in isolation. Ensure that all configurations use consistent hyperparameters (depth, width, learning rate, batch size) where applicable.

Benchmarking Configuration 5 (BLT)

Configuration 5 (C5) replaces the standard subword tokenizer with a simplified Byte Latent Transformer (BLT) architecture. Instead of using a vocabulary, C5 will feed raw bytes into a local encoder to create patch representations, process those through the global transformer, and decode them using a local byte-level decoder.

When comparing C5 against the base C1 model, pay special attention to:

- Training speed and computational overhead.
- Peak GPU memory usage during training.
- Overall sequence reconstruction performance on the test set.

4 Evaluation Metrics

For all experiments, generate your numbers using greedy decoding so results are consistent across models. Use the following metrics to evaluate your results:

- Bit-Level Accuracy: The percentage of exact bit matches.
- Sequence Accuracy: The percentage of sequences that are perfectly reconstructed.
- Levenshtein Distance: The edit distance between the target and predicted outputs.
- BLEU and ROUGE Scores: Standard n-gram overlap metrics (for tokenized models only).

5 Deliverables

Upload a single ZIP file named `<rollnumber>_assignment1.zip` to Moodle. Your folder structure should look exactly like this:

```
<rollnumber>_assignment1/
|-- src/
|   |-- models/
|   |   |-- attention.py    # MHA and GQA
|   |   |-- positional.py   # Sinusoidal and RoPE
|   |   |-- norm.py         # LayerNorm and RMSNorm
|   |   |-- blt.py          # Local Encoder/Decoder for BLT
|   |-- dataset.py          # Tokenized vs Token-Free Loaders
|   |-- train.py            # Main training loop with WandB
|   |-- utils.py            # Metrics and plots
|-- outputs/                # Saved plots and logs
|-- README.md               # Setup instructions and HF links
-- Report.pdf               # Your final report
```

Report Guidelines

Your PDF report should be a maximum of 6 pages (excluding references and links). Use 12pt Times New Roman with 1-inch margins. Include tables and plots that discuss the results of your ablation study (C1 through C5). Discuss what each single-component change improved or worsened compared to the base model, paying special attention to the tradeoffs in memory and accuracy introduced by the token-free BLT approach.

6 Grading Scheme

Component	Marks
Architectural Modules (MHA, GQA, RoPE, RMSNorm, BLT modules)	15
Implementation of 5 Configurations (C1-C5)	20
Experimental Results, WandB Logs & HuggingFace Links	8
Report Quality, Analysis & Directory Structure	7
Viva / Code Defense	50
Total	100